



DATA STRUCTURES

CONTENTS



21. Binary Search Trees-I

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Tree ADT. In this module we will discuss one very important type of binary tree that is binary search tree ADT.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the concept of Binary Search Trees
- To discuss the Binary Search Tree Search
- To describe Traversal of Binary Search Trees – the In-order traversal
- To explain the operations of finding the Successor and Predecessor of a node in a Binary Search Tree

21.1 Introduction

Binary Search Trees (BST) or Lexicographic Trees are a type of Binary Trees with a special organization of data. This organization of the nodes of the binary tree makes searching for an element faster of the $O(\log n)$ and for some types of balanced binary search trees (we will discuss this in later modules) insertions and deletions also has a time complexity of the $O(\log n)$. However in general insertions and deletions in BST have a complexity of the $O(h)$ where h is the height of the tree. The properties of BST are defined in terms of the maximum number of leaves, maximum number of nodes N and the average depth for the N nodes.

A binary search tree is a binary tree with a special property called the BST-property, which is defined as follows:

- For a tree or sub-tree rooted at X ,
 - All keys to the left of X are lesser than the key at X
 - All keys to the right of X are greater than the key at X
- This property is true for every sub-tree of the BST
- We can assume that the keys of a BST are pairwise distinct

In other words all items in the left subtree are less than the root K , and all items in the right subtree are greater than the root K (Figure 21.1).

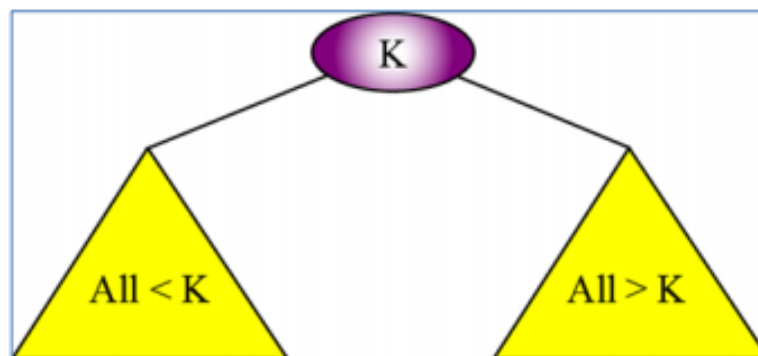


Figure 21.1 A Binary Search Tree

21.2 Binary Search Tree Property

All the stored keys in a binary search tree must satisfy the *binary search tree*

property.

All nodes K in left sub-tree of R , will be such that $key[K] \leq key[R]$.

Similarly all nodes in right sub-tree of R , will be such that $key[K] \geq key[R]$.

In other words in a BST, the value stored at the root of a sub-tree is *greater* than any value in its left subtree and *less* than any value in its right subtree (Figure 21.2).

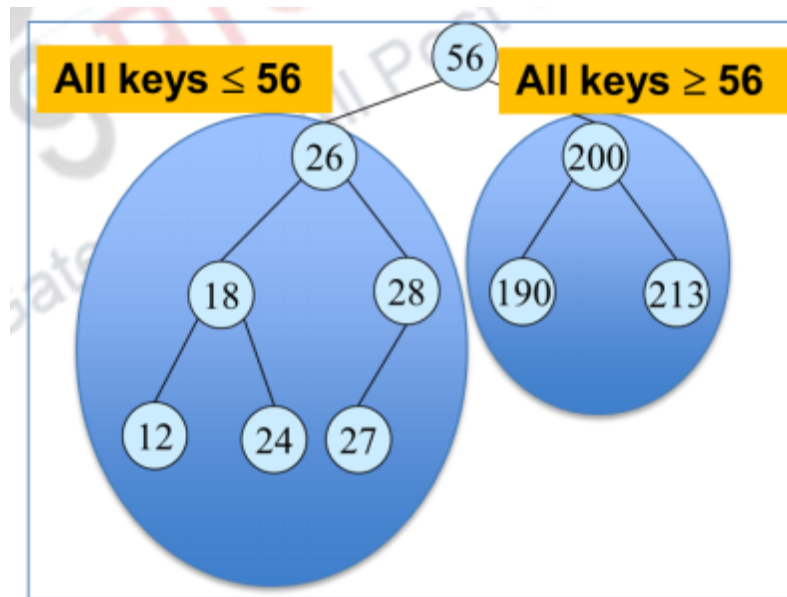


Figure 21.2 Example of a Binary Search Tree

21.3 Other properties of Binary Search Trees

The minimum node is identified as the leftmost node, i.e. the farthest node you can reach by following only left branches. Similarly the maximum node is identified as the rightmost node, i.e. the farthest node you can reach by following only right branches. Hence the smallest element in a binary search tree is the **leftmost element** and the largest element is the **rightmost element** (Figure 21.3). The same set of keys can result in different BSTs depending on the order in which the keys are inserted into the BST (Figure 21.4). The search time of a node in a tree depends on its depth. In the case of a binary search tree, the maximum depth in the skewed case with the least or the largest key at the root is N where N is the number of nodes. However the minimum depth is $\log N$.

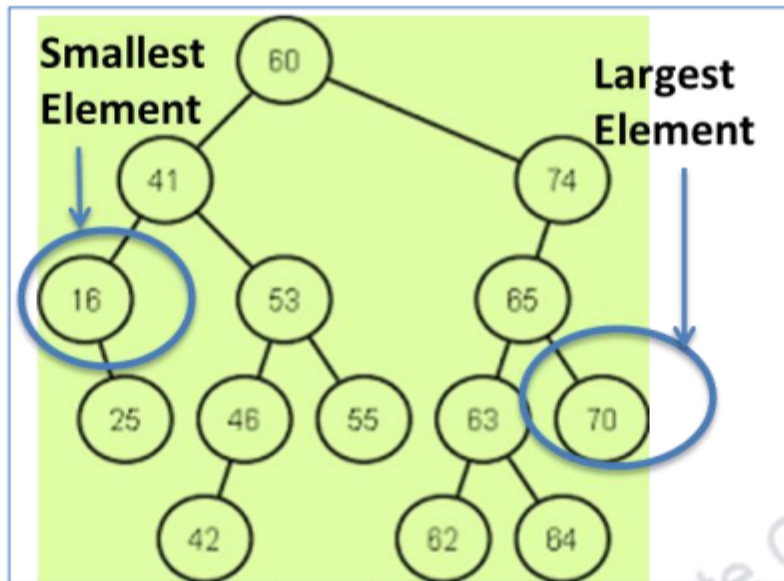


Figure 21.3 Smallest and Largest Element of a Binary Search Tree

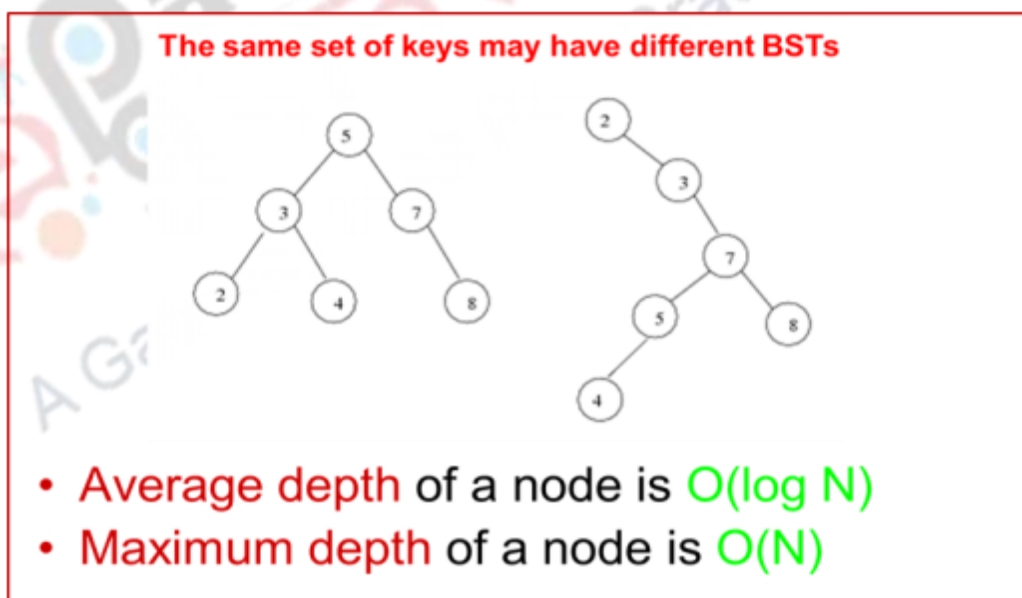


Figure 21.4 Different BSTs for the same set of Keys

Let us now see BST examples and examples that are not BSTs (Figure 21.5). The first example shows a BST with keys being names so here the lexicographic order is relevant while the next example shows a BST with keys being numbers and hence

numeric order is followed. The third example shows a tree that is not a BST since the left sub-tree is not binary and the right sub-tree does not satisfy the binary search property.

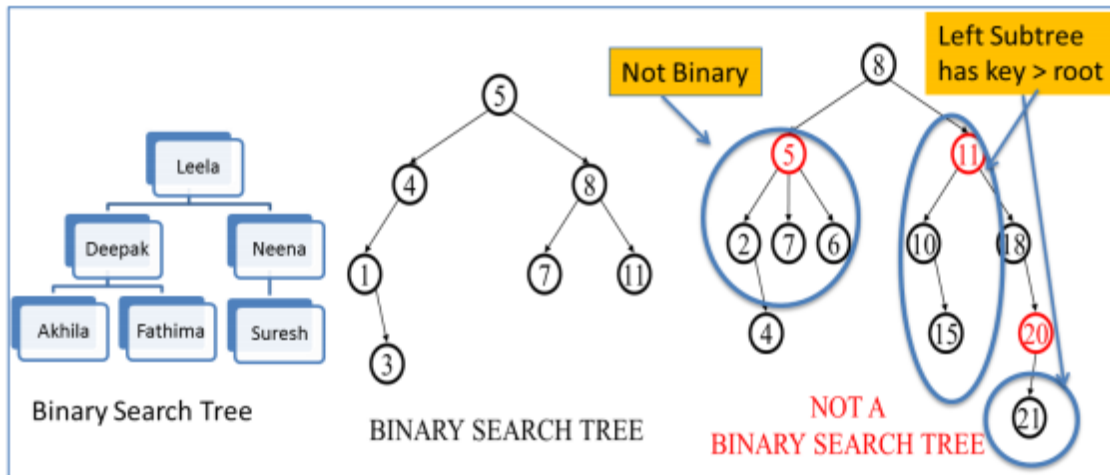


Figure 21.5 Examples and Counter Examples of Binary Search Trees

Sometimes duplicate entries are allowed in binary search trees in which case they are always placed in the right subtree (Figure 21.6).

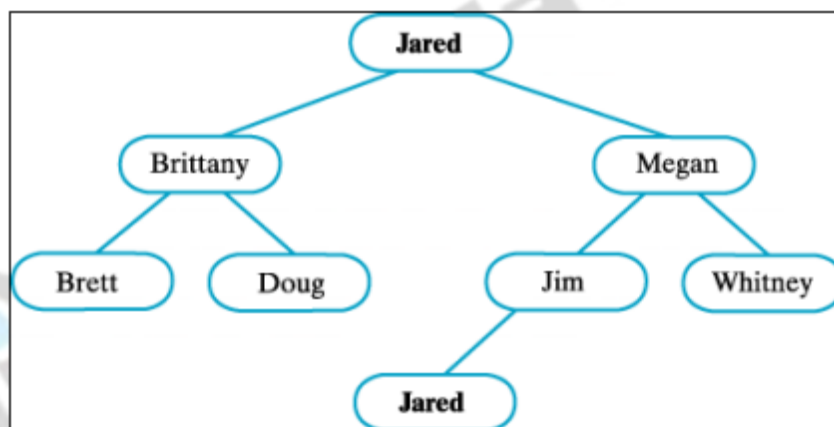


Figure 21.6 Binary Search Tree with Duplicate Entries

Binary Search Trees data structures that can support dynamic set operations like search, minimum, maximum, predecessor, successor, insert, and delete and hence

are appropriate for building dictionaries and priority queues. All the basic operations associated with binary search trees have maximum complexity of the $O(h)$ where h is the height h of the tree.

21.4 Ordered Dictionaries

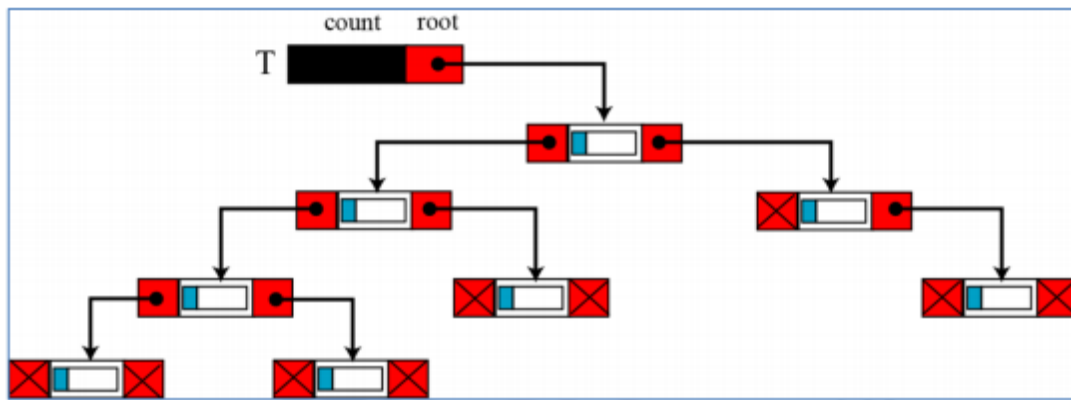
When binary search trees are used to represent dictionaries the keys are assumed to come from a total order. Remember that a total order relation satisfies the reflexive, anti-symmetric and transitive properties. Examples of total order relations include $\leq$ and $\geq$ for numerical order and we can define $a < b$ if a comes before b in the lexicographic order. Some of the operations associated with the dictionary are:

- `first()`: the first entry in the ordering of the dictionary
- `last()`: last entry in the ordering of the dictionary
- `successors(k)`: iterator of entries with keys greater than or equal to k ; where the order is increasing
- `predecessors(k)`: iterator of entries with keys less than or equal to k where the order is decreasing

21.5 Binary Search Tree (BST) ADTs

The ADT for a binary search tree is similar to the ADT for a general linear list with the same operations. However searching a BST is more efficient than searching a linear list. While a general linear list uses sequential searching, BSTs use a version of binary search.

BSTs can be implemented using either arrays or linked lists. However, linked list structures are more common and more efficient. The implementation uses nodes with two pointers, left and right with the node containing the data element (Figure 21.7)



The Binary search tree is represented by a linked data structure of nodes. Here $root(T)$ points to the root of tree T . In one implementation each node contains the fields *-key*, *left* – pointer to left child, *right* – pointer to right child and *p* – pointer to parent. $p[root[T]] = NIL$ (optional).

Some of the typical operations of the ADT binary search tree are

- **Retrieving** the item with a given search key from a binary search tree
- **Traversing** the items in a binary search tree in preorder, in-order, or post-order
- **Inserting** a new item into a binary search tree such that BST property is maintained
- **Deleting** the item with a given search key from a binary search tree such that BST property is maintained

BST allows us to implement the following accessing operations

- Search(element) – search for a particular element in the BST
- add(element) – add or insert an element in its proper place in the BST
- getHeight – Get the height of the BST
- getMin, getMax – get the minimum / maximum key from the BST
- removeMin, removeMax – remove the minimum / maximum key from the BST and restore the BST property

- `remove(element)` – remove the element from the BST and restore the BST property
- `printInOrder`, `printPreOrder`, `printPostOrder` – traverse the BST and print the elements in the required order

21.6 Searching BST

There are basically three search algorithms possible in a BST – finding the smallest node, finding the largest node and finding a requested node (BST search). For example in Figure 21.8 suppose we are searching for a node. If we are searching for 15, then we are done. If we are searching for a key < 15 , then we should search in the left subtree. If we are searching for a key > 15 , then we should search in the right subtree.

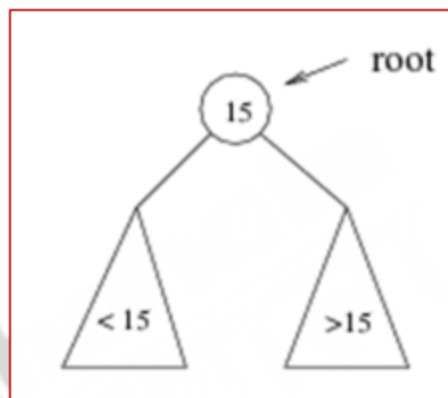


Figure 21.8 Searching for a Key

21.6.1 Finding Minimum and Maximum

In order to find the minimum element in the BST, we follow *left* children until we reach null. Similarly in order to find the maximum element in the BST, we follow *right* children until we reach null (Figure 21.9).

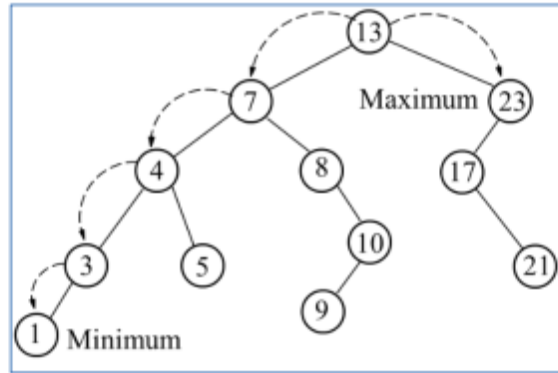


Figure 21.9 Minimum and Maximum Elements in the BST

The binary-search-tree property guarantees that the minimum key is located at the left-most node and the maximum key is located at the right-most node. The algorithm to find these elements is given in Figure 21.10 where x is a pointer to a tree node.

Tree-Minimum(x)

1. **while** $left[x] \neq NIL$
2. **do** $x \leftarrow left[x]$
3. **return** x

Tree-Maximum(x)

1. **while** $right[x] \neq NIL$
2. **do** $x \leftarrow right[x]$
3. **return** x

Figure 21.10 Finding Minimum and Maximum Elements

Binary Search algorithm of an array of *sorted* items reduces the search space by one half after each comparison (Figure 21.11). Many of the other operations associated with BST use the search operation.

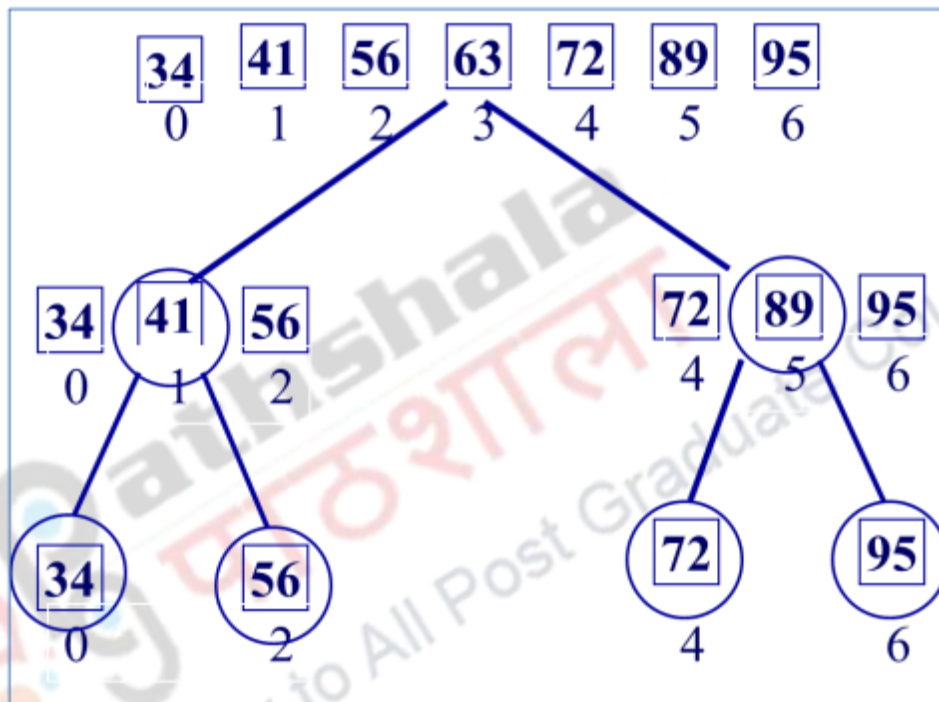


Figure 21.11 Reducing Search Space by Half

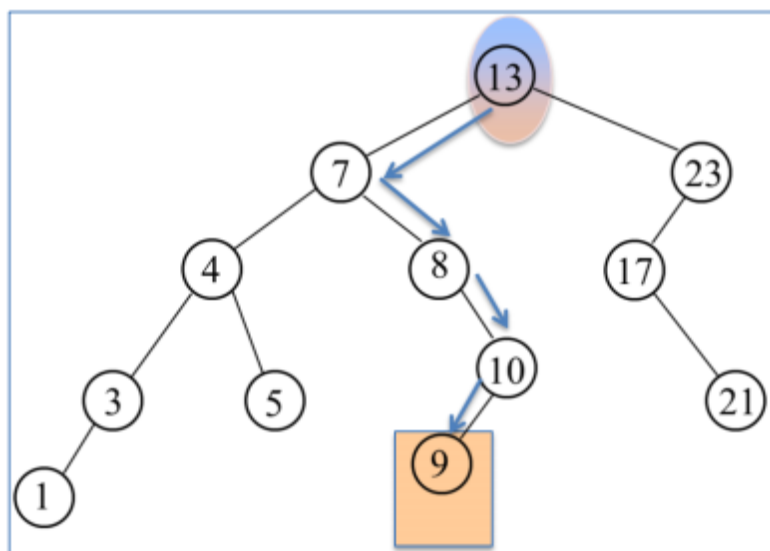


Figure 21.12 Searching for the Element 9

As shown in Figure 21.12 when we want to search for 9, we compare 9 with 13 (root); and go to left subtree. Then we compare 9 with 7; and go to right subtree, then we

compare 9 with 8 and again go to right subtree. Then we compare 9 with 10 and go to left subtree. Finally we compare 9 with 9 and we find the element we searched for. At this point if we do not find the element then the element is not in the BST.

Now let us understand both the recursive and iterative algorithms for searching an element in the BST. The recursive algorithm is easier to understand as it look for the key and if the key to be found is less than root we call Tree-Search of the left subtree else if the key is greater than the root we call Tree-Search of the right subtree. We recursively do this till we find the key or the key is not in tree. In the iterative case we use a pointer x (pointer to the tree node) to traverse the tree (Figure 21.13). The iterative tree search is more efficient on most computers. The running time is $O(h)$ where h is the height of the tree.

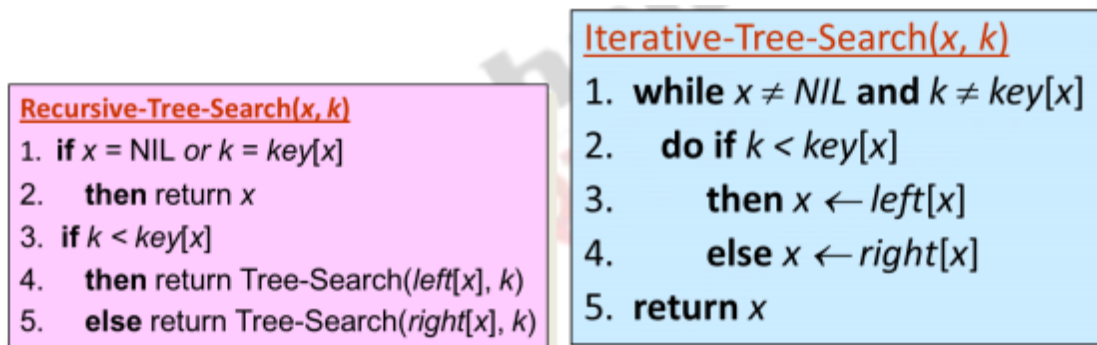


Figure 21.13 Tree Search Algorithms

21.7 In order Traversal of a BST

A very interesting property of a BST is that if we apply the inorder traversal, the elements that are visited are sorted in ascending order. When we print the elements of a binary search tree in the order of the recursive in-order traversal (or walk) of the tree, the binary-search-tree property ensures that the keys are printed in increasing order. The recursive algorithm is given in Figure 21.14 where x is the node from which we start the traversal. The Inorder traversal of a BST is similar to the in-order traversal of a binary tree described in the previous module.

Inorder-Tree-Walk (x)

1. **if** $x \neq \text{NIL}$
2. **then** Inorder-Tree-Walk($\text{left}[x]$)
3. print $\text{key}[x]$
4. Inorder-Tree-Walk($\text{right}[x]$)

Figure 21.14 Inorder Traversal of a BST

21.7.1 Successor and Predecessor of a Node

The successor and predecessor of a node is defined in terms of the in-order traversal of the binary search tree. Successor of node x is defined as the node y such that the *key* of y is the smallest key (left most node) that is greater than $\text{key}[x]$. The successor of the largest key is NIL. In this case searching for the successor needs to consider two cases. If node x has a non-empty right subtree, then x 's successor is the minimum key in the right subtree of x . However if the node x has an empty right subtree, we move up the tree (move up through right children). Now we encounter keys that are smaller than x . We need to first find the lowest ancestor of x . If node x has an empty right subtree and x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x .

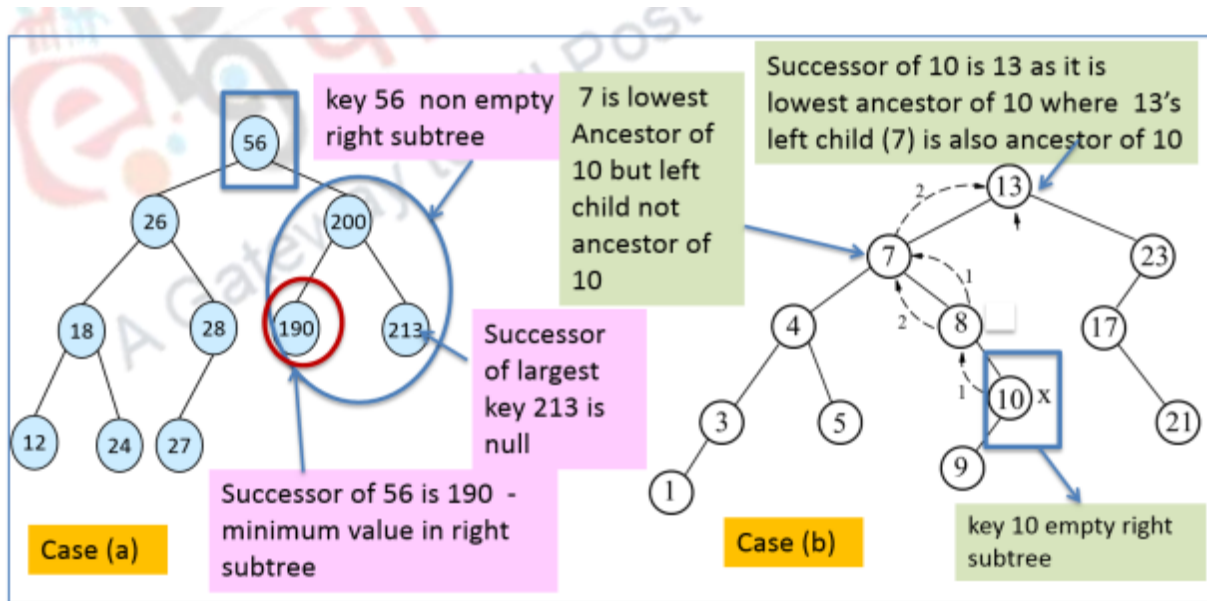


Figure 21.15 Successor of a Node

Let us consider Case(a) of Figure 21.15. Suppose we want to find successor of 56. Now 56 has a non-empty right subtree and hence the successor of 56 is the minimum value in the right subtree – in this case 190. Now considering Case(b), 10 whose successor we want to find has an empty right subtree. Therefore we move left up the tree and find its successor 13. The algorithm for finding the successor is given in Figure 21.16 and for finding predecessor which is symmetric is given in Figure

21.17. Both the algorithms have a running time of $O(h)$ where h is the height of the tree.

Tree-Successor(*x*)

```
♦   if right[x] ≠ NIL
2.   then return Tree-Minimum(right[x])
3.   y ← parent[x]
4.   while y ≠ NIL and x = right[y]
5.   do x ← y
6.   y ← parent[y]
7.   return y
```

Figure 21.16 Finding Successor

Tree-Predecessor(*x*)

```
♦   if left[x] ≠ NIL
2.   then return Tree-Maximum(left[x])
3.   y ← p[x]
4.   while y ≠ NIL and x = left[y]
5.   do x ← y
6.   y ← p[y]
7.   return y
```

Figure 21.16 Finding Predecessor

Summary

- Explained the concept of Binary Search Trees
- Discussed Searching operation of Binary Search Trees
- Described In order Traversal of Binary Search Trees
- Explained the Successor and Predecessor operations associated with Binary Search Trees

you can view video on Binary Search Trees-I



Web Links
https://www.cs.princeton.edu/~rs/AlgsDS07/08BinarySearchTrees.pdf
http://algs4.cs.princeton.edu/32bst/
http://www.cs.swarthmore.edu/~newhall/unixhelp/Java_bst.pdf
https://www.cs.rochester.edu/~gildea/csc282/slides/C12-bst.pdf
http://www.learnalgorithms.in/data_structures/tree.php
http://dcslab.snu.ac.kr/courses/intro200
http://www.utdallas.edu/~daescu/cs3345spr12.html
users.cis.fiu.edu/~taoli/class/COT5407-F09/lecture/btrees.ppt
http://uet.vnu.edu.vn/~chauttm/dsa2011w/

Supporting & Reference Materials
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Robert Sedgewick, Kevin Wayne "Algorithms", Addison-Wesley Professional, 4th Edition, 2011
Thomas H. Cormen, et.al, "Introduction to Algorithms" The MIT Press; 3rd edition, 2009
Anany Levitin "Introduction to the Design and Analysis of Algorithms", Pearson; 3rd edition 2011